Path Finding (A* Algorithm)

8 Steps to Finding the Shortest Path (Assuming you have all data setup)

1. Set start point G to 0
2. Add start point to list of **OPEN** Nodes (Nodes to be checked for path acceptance)

LOOP the following steps until:

- a path is found (the target is added to the **CLOSED** list)
  OR
- a path is impossible (No more Nodes to check in the **OPEN** list)

3. Find the Node in the **OPEN** list with the **SMALLEST** F cost, **REMOVE** it from the **OPEN** and **ADD** it to the **CLOSED** list
4. If the added Node is the Target Node, **STOP** looping, the Path is found.
5. The Node added to the **CLOSED** list is our current Node (curNode), Go through each of curNode's adjacent tiles and:
   a. **IGNORE** it if it is already in the closed list or is not a walkable Node (e.g. A Wall)
   b. If it is **NOT** in the **OPEN** list, Set its parent to the curNode, recalculate its G, H and F costs and **ADD** it to the **OPEN** list
   c. If it IS in the **OPEN** list, compare its current G cost with its potential new G cost. If the new G cost is less (better), **set** its parent to curNode, **set** its G to the new G and **recalculate** its F Cost
   d. If you are **SORTING** the **OPEN** list by F, this would be the time to redo the sort (not needed for small maps)
6. If the **OPEN** list is empty, **STOP** looping, **NO** path is possible

END LOOP

7. If a path was found (Target is in the **CLOSED** list)
   a. Beginning with the target Node, add it to the front of a list of **PATH** Nodes
   b. Then **ADD** its parent to the **PATH** list
   c. Continue to do this until the parent does not exist (the only Node without a parent is the start Node)
8. Return the **PATH** list (which will either be empty of full of path grid coordinates)